01 | صفحه عنوان و چکیده اجرایی

عنوان پروژه: Distributed Banking Platform — Sharding, Replication, 2PC, Live Re-Sharding, Autonomous Self-Healing

نام دانشجو: ..... استاد / بررسی‌کننده: .....

مخزن GitHub: https://github.com/USERNAME/distributed-banking-platform

مخزن Docker Hub: https://hub.docker.com/r/USERNAME/distributed-banking-app

چکیده

هدف پروژه طراحی و ارزیابی سامانه بانکی توزیع‌شده‌ای است که داده‌های حساب را بین پایگاه‌های داده مستقل تقسیم کند، درخواست‌ها را میان Replicaهای سالم توزیع کند، تراکنش‌های بین پایگاه داده را با Two-Phase Commit هماهنگ سازد و پس از خرابی سرویس‌ها به وضعیت سازگار بازگردد. سیستم روی یک میزبان با Docker Compose اجرا می‌شود و از Python، FastAPI و PostgreSQL استفاده می‌کند.

علاوه بر نیازمندی‌های پایه، دو قابلیت پیشرفته بخشی از دامنه اصلی پیاده‌سازی‌اند: مهاجرت زنده محدوده حساب‌ها از Shard B به C با ثبت تغییرات و Cutover امن؛ و Controller خودکار برای کشف خرابی، ساخت کانتینر جدید و بازیابی پس از Crash بدون ایجاد Replica تکراری.

دامنه و وضعیت ارزیابی

معیار اصلی	محور
مالکیت یکتای حساب و حداقل 2 Replica برای هر Shard	Sharding / Replication
پرهیز از Lost Update؛ تصمیم پایدار و بازیابی تراکنش	ACID و 2PC
Snapshot، Replay، Epoch، Fencing، Crash Recovery	Bonus I (+12%)
Controller سفارشی، Replacement و Recovery بدون Duplicate	Bonus II (+10%)

وضعیت نتایج: تا زمان تنظیم این گزارش، خروجی عملی ارسال‌شده از آزمون Stage 14 نشان می‌دهد Shard Map هنوز محدوده A1501–A2000 را متعلق به B می‌داند؛ پس آزمون postcutover موفق نشده و تکمیل عملی Bonus I باید دوباره بررسی شود. موفقیت تست‌های واحد یا ساختاری جایگزین این تأیید نیست.

02 | معماری نرم‌افزار و استقرار

لایه Client صرفاً از APIهای Gateway استفاده می‌کند. Gateway با خواندن Shard Map از Metadata، مالک حساب را مشخص و سپس از Registry نسخه‌دار، Replica سالم همان Shard را انتخاب می‌کند. Replicaها مسئول اجرای عملیات حساب هستند و همه Replicaهای یک Shard به پایگاه داده مشترک همان Shard متصل می‌شوند؛ بنابراین موجودی در حافظه Replicaها مرجع نهایی نیست.

اجزای کلیدی

سرویس	وظیفه و داده پایدار
Gateway	مسیریابی پویا، Round Robin، Health Check و استعلام وضعیت
Shard A / B / C	Replicaهای سرویس بانکداری و پایگاه داده مستقل PostgreSQL
Coordinator	شناسه تراکنش، رأی PREPARE، تصمیم قطعی و Recovery
Metadata DB	Shard Map، Epoch، Replica Registry، Migration Job و رویدادها
Migration Manager	Snapshot، خواندن ژورنال، Replay، Final Catch-up و Cutover
Self-Healing Controller	شناسایی خرابی، مدیریت نسل‌ها و ساخت کانتینر جایگزین

مسیر درخواست

Client → Gateway → (Metadata: Shard Map + Replica Registry) → Replica سالم → Shard DB. برای انتقال میان Shardها، Gateway درخواست را به Coordinator می‌سپارد؛ Coordinator با Participantهای مرتبط و Metadata تعامل می‌کند. در مسیر تغییر مالکیت، Migration Manager با هر سه دیتابیس و Shard Map هماهنگ می‌شود.

زیرساخت استقرار

پایگاه‌های داده A، B، C و Metadata دارای Volume پایدار جداگانه هستند. PostgreSQL 16 با max_prepared_transactions=100 برای Participantها تنظیم شده است. شبکه داخلی Compose ارتباط سرویس‌ها را فراهم می‌کند. اجرای Docker روی یک میزبان شرط پروژه است؛ بنابراین معماری تحمل خرابی میزبان یا دیتابیس فیزیکی را ادعا نمی‌کند.

محدودیت عملیاتی: اتصال Docker Socket صرفاً برای Controller و در محیط توسعه مورداعتماد در نظر گرفته شده است؛ این دسترسی معادل اختیار بالای مدیر میزبان است.

03 | تقسیم‌بندی حساب‌ها، Shard Map و مسیریابی

در آغاز، تخصیص حساب‌ها بر پایه محدوده عددی شناسه انجام می‌شود. شناسه A0500 به Shard A و شناسه A1500 به Shard B تعلق دارد. این انتساب قطعی است و داده کامل حساب‌ها روی تمام دیتابیس‌ها کپی نمی‌شود. درخواست‌های یک حساب تنها به Replicaهای Shard مالک هدایت می‌شوند.

نقشه مالکیت پیش و پس از مهاجرت

بازه حساب	پیش از **Cutover**	پس از **Cutover**
A0001–A1000	A	A
A1001–A1500	B	B
A1501–A2000	B	C

جدول shard_map در Metadata محدوده‌های فعال را به همراه epoch نگه می‌دارد. با هر تغییر مالکیت، routing_epoch به‌صورت افزایشی تغییر می‌کند؛ سپس Gateway Epoch مورد انتظار را همراه با درخواست به Replica ارسال می‌کند. Replica علاوه بر شناسه Shard، Fence محلی حساب را بررسی می‌کند.

اهمیت کنترل در مسیر نوشتن

صرف تغییر Gateway برای جلوگیری از درخواست کهنه کافی نیست؛ ممکن است درخواست با Route قدیمی هنوز در شبکه باشد یا مستقیماً به Replica فرستاده شود. بنابراین بررسی مالکیت و Epoch باید در مسیر تراکنش بانکی، هم‌زمان با قفل‌های دیتابیس، اعمال شود. پس از انتقال ایمن به C، هر نوشتن با Epoch قبلی روی B باید بدون تغییر موجودی و نسخه حساب، با خطای تعارض رد شود.

شواهد قابل ارائه

- آزمون Stage03 برای جداسازی داده‌ها و مالکیت اولیه A/B؛ Stage12 stale برای رد Route قدیمی.
- آزمون Stage14 postcutover برای سه بازه جدید، شش Replica سالم، وجود داده فعال در C و نبود تراکنش آماده حل‌نشده.

یادآوری: نتیجه فعلی postcutover در محیط گزارش‌شده FAIL است؛ زیرا Shard Map هنوز B را مالک کل A1001–A2000 نشان می‌دهد. این وضعیت باید از مسیر رسمی Cutover اصلاح و دوباره آزمایش شود.

04 | عملیات بانکی، ACID و کنترل هم‌زمانی

عملیات پایه شامل موجودی، واریز، برداشت و انتقال درون‌شاردی است. برای جلوگیری از Lost Update، کنترل هم‌زمانی در PostgreSQL قرار دارد، نه در قفل‌های حافظه محلی Replica؛ زیرا A1 و A2 حافظه مشترک ندارند. برداشت باید موجودی کافی را بررسی کند و انتقال درون یک Shard در یک تراکنش SQL، هر دو حساب را اتمیک تغییر دهد.

قواعد صحت

- Atomicity: یا هر دو طرف انتقال داخلی ثبت می‌شوند یا هر دو Rollback می‌شوند.
- Consistency: موجودی منفی یا تغییر غیرمجاز مالکیت نباید پذیرفته شود.
- Isolation: عملیات هم‌زمان با قفل سطری و ترتیب مناسب از Lost Update جلوگیری می‌کنند.
- Durability: داده حساب و Journal پس از COMMIT در PostgreSQL پایدار می‌شوند.

Idempotency و Version

هر درخواست تغییردهنده پول دارای request_id است. شناسه و مشخصات درخواست ثبت می‌شود تا Retry یک پیام، دوباره موجودی را تغییر ندهد. استفاده از همان شناسه با پارامترهای ناسازگار باید با خطای تعارض رد شود. Version حساب امکان اعتبارسنجی و Replay ترتیبی تغییرات را فراهم می‌کند.

روش ارزیابی

تست Stage05 چند عملیات هم‌زمان واریز/برداشت را از Replicaهای مختلف اجرا می‌کند و موجودی نهایی را با نتیجه مورد انتظار مقایسه می‌کند. برای یک انتقال جبرانی، ابتدا واریز و سپس برداشت متناظر با شناسه‌های متفاوت اجرا می‌شود؛ با پایان موفق، اثر خالص پولی صفر است؛ اما Version و Journal تغییر می‌کنند. در صورت Crash وسط تست، نباید با شناسه جدید و بدون استعلام تراکنش، عملیات جبرانی ساخت.

برتری روش: قفل سطری در محل داده معتبر اعمال می‌شود و در برابر تغییر Replica فعال نیز خاصیت سازگاری حفظ می‌شود.

05 | پروتکل تراکنش توزیع‌شده 2PC

برای انتقال میان Shardهای مستقل، دو به‌روزرسانی SQL پیاپی کافی نیست. Coordinator یک شناسه یکتا اختصاص می‌دهد، Participantهای منبع و مقصد را مشخص می‌کند، سپس دو فاز Prepare و Decision را پیش می‌برد. Participantها از PREPARE TRANSACTION واقعی PostgreSQL استفاده می‌کنند و تا تصمیم نهایی، تراکنش را آماده ولی Commitنشده نگه می‌دارند.

توالی پیام‌ها (خلاصه Sequence Diagram)

- ۱) Client → Gateway → Coordinator: Transfer(request_id, from, to, amount)
- ۲) Coordinator → Metadata: ثبت INITIATED و اطلاعات Participantها
- ۳) Coordinator → Shard A: PREPARE؛ پاسخ YES با حالت پایدار
- ۴) Coordinator → Shard B/C: PREPARE؛ پاسخ YES یا NO
- ۵) Coordinator → Metadata: COMMIT_DECIDED یا ABORT_DECIDED
- ۶) Coordinator ← هر Participant: COMMIT PREPARED / ROLLBACK PREPARED
- ۷) Participant → Coordinator: ACK؛ Metadata COMPLETED
- ۸) Client → Gateway: استعلام نتیجه؛ COMMITTED فقط پس از همه ACKها

شکست و قاعده تصمیم

اگر یکی از Participantها رأی NO بدهد، Coordinator تصمیم ABORT را به‌شکل پایدار ثبت و Preparedهای قبلی را Rollback می‌کند. اگر همه رأی YES بدهند، تصمیم COMMIT پیش از ارسال پیام‌های فاز دوم در Metadata ثبت می‌شود. پس از ثبت COMMIT_DECIDED، قطعی موقت یک Shard مجوز تغییر تصمیم به ABORT نیست؛ تراکنش می‌تواند تا بازیابی سرویس در وضعیت PENDING بماند.

در یک انتقال نمونه A1500 → A0500 به مبلغ 300، موجودی 1000/500 باید به 700/800 تبدیل شود؛ مثال آموزشی است و نتیجه اجرای فعلی سیستم را اثبات نمی‌کند.

06 | تصمیم پایدار، بازیابی **Crash** و وضعیت نامشخص

Coordinator وضعیت هر تراکنش را در جدول Metadata ثبت می‌کند: INITIATED، PREPARING، PREPARED، COMMIT_DECIDED، ABORT_DECIDED و COMPLETED. Recovery Worker پس از Restart پرونده‌های حل‌نشده را بررسی و همان تصمیم پایدار را ادامه می‌دهد. Participantها پیام تکراری COMMIT/ABORT را با ثبت Ledger به‌شکل بی‌اثر پردازش می‌کنند.

سناریوی خرابی کنترل‌شده مرحله 07

در سناریو، هر دو Shard آماده می‌شوند، Coordinator تصمیم COMMIT را ذخیره می‌کند، Shard A را Commit کرده و ACK آن را ثبت می‌کند. سپس Coordinator عمداً با خروج ناگهانی متوقف می‌شود، در حالی که Shard B هنوز PREPARED است. Gateway از Metadata نتیجه PENDING ارائه می‌کند. پس از Restart، Coordinator تصمیم ثبت‌شده را بازیابی، COMMIT B را کامل و تراکنش را COMPLETED می‌کند.

معیارهای اثبات

- پیش از Restart: COMMIT_DECIDED، یک ACK ثبت‌شده، تراکنش Prepared باقی‌مانده و وضعیت PENDING.
- پس از Restart: دو ACK پایدار، نبود Prepared باقی‌مانده و عدم تکرار اثر مالی.
- سناریوی رأی Rollback: NO تمام Participantهای آماده‌شده و حفظ مجموع موجودی.

محدودیت **2PC**

پروتکل 2PC ماهیت Blocking دارد. اگر Participant یا Coordinator برای مدت طولانی در دسترس نباشد، تراکنش ممکن است PENDING باقی بماند و قفل‌های Prepared Transaction را نگه دارد. این طرح جایگزین Consensus یا Replication خود PostgreSQL نیست و طبق صورت تمرین، Failover پایگاه داده الزامی نشده است.

شواهد ارائه: خروجی‌های واقعی verify_stage07 با سناریوهای crashed/recovered، وضعیت pg_prepared_xacts و شناسه request_id. در نبود خروجی واقعی، سناریو صرفاً «طراحی و تست قابل اجرا» محسوب می‌شود.

Cutover و Live Re-Sharding، Snapshot، Replay | 07

Bonus I انتقال آنلاین A1501–A2000 از B به C است. در زمان کپی اولیه، کاربران باید همچنان امکان عملیات روی حساب‌های مبدأ را داشته باشند. برای این هدف Snapshot سازگار با PostgreSQL گرفته شده و تغییرات پس از Snapshot در Journal برنامه ثبت و به ترتیب Commit با Checkpointهای پایدار بازپخش می‌شوند. این سازوکار ژورنال سطح برنامه است و ادعای استفاده از PostgreSQL WAL Logical Decoding ندارد.

ماشین حالت عملیاتی

مرحله	قانون حفظ صحت
SNAPSHOTTING	ثبت Snapshot پایدار؛ مالک هنوز B
CATCHING_UP	Replay گروهی و Idempotent در جدول‌های موقت C
FENCING	بستن نوشتن B روی محدوده در حال انتقال
FINAL_REPLAY	رفع فاصله نهایی نسخه و موجودی با حل 2PCهای مرتبط
MAP_ACTIVATED	تغییر اتمیک Shard Map و افزایش Epoch در Metadata
VERIFYING	فعال‌سازی C به‌عنوان تنها Owner و رد Route قدیمی

ترتیب Cutover عمداً Fail-Closed است: ابتدا B غیرقابل نوشتن می‌شود؛ سپس آخرین تغییرات روی C اعمال و صحت آن‌ها بررسی می‌شود؛ بعد C در حالت آماده ولی بدون نوشتن قرار می‌گیرد؛ تنها پس از ثبت تغییر Shard Map و Epoch، نوشتن C مجاز می‌شود. وقفه کوتاه پذیرفته است، اما هم‌زمان دو مالک قابل نوشتن نباید وجود داشته باشند.

سازگاری با 2PC و درخواست قدیمی

تراکنش‌های Prepared مرتبط پیش از تغییر مالکیت باید حل شوند یا Cutover متوقف بماند؛ COMMIT تصمیم‌گرفته‌شده هرگز دور ریخته نمی‌شود. علاوه بر آن، تاریخچه request_idهای B به Metadata آرشیو می‌شود تا Retry بعد از مهاجرت در C دوباره پول جابه‌جا نکند.

وضعیت فعلی مشاهده‌شده: postcutover هنوز نقشه C را ندیده؛ برای تأیید امتیاز کامل باید Cutover، stale fencing و Crash Recovery چهار نقطه مرحله 14 با Docker پاس شوند.

08 | Self-Healing Controller و بازیابی خودش

Bonus II شامل Controller سفارشی است که خرابی Replica را شناسایی می‌کند، تعداد Replicaهای سالم هر Shard را در سطح تنظیم‌شده نگه می‌دارد و با Docker Engine API کانتینر جایگزین می‌سازد. استفاده از Docker restart policy، Kubernetes، Swarm یا ابزار آماده Self-Healing در این مسیر ممنوع است؛ بنابراین منطق Reconciliation در سرویس Python پیاده شده است.

الگوریتم بازیابی

- گرفتن قفل رهبری Advisory در PostgreSQL و خواندن Replica Registry و desired_replica_count.
- بررسی Health و نسل واقعی Container بر اساس Label و پاسخ HTTP؛ اجتناب از Replace در خرابی مشترک دیتابیس.
- رزرو Generation جدید به‌صورت پایدار؛ ساخت کانتینر با نام قطعی؛ Start و Registration در Metadata.
- بررسی اینکه تنها یک کانتینر فعال برای Slot منطقی وجود داشته باشد و Gateway آدرس نسل تازه را بخواند.

چهار نقطه Crash Controller

بازیابی مورد انتظار	نقطه
استفاده از همان Generation رزرو شده	AFTER_RESERVE
پیدا کردن Container موجود، بدون Create تکراری	AFTER_CREATE
ثبت همان Container اجراشده به‌عنوان سالم	AFTER_START
حفظ Replica سالم و جلوگیری از Generation اضافی	AFTER_HEALTHY

در تست‌های واقعی باید یک Replica زیر بار متوقف شود، Gateway همچنان از Replica دیگر استفاده کند و Controller پس از Restart به سهمیه سالم بازگردد. پیام HTTP 200 از /health به‌تنهایی کافی نیست؛ حلقه Reconciliation باید واقعاً پیش برود. در نسخه اولیه، ناسازگاری دسترسی dict_row با locked[0] سبب KeyError شده بود که اصلاح این خط برای عملکرد کنترلر ضروری است.

محدودیت امنیتی: دسترسی به docker.sock قدرت بالایی روی میزبان ایجاد می‌کند؛ استفاده صرفاً در محیط آزمایش امن توصیه می‌شود.

09 | طرح آزمون، معیار پذیرش و شواهد

پروژه دارای هفت آزمایش اجباری است. هر آزمایش باید با شناسه، دستور اجرا، خروجی PASS/FAIL، زمان و در صورت عملیات مالی request_id ثبت شود. برخی تست‌ها پیش از Cutover معتبرند و برخی تنها پس از فعال‌شدن C؛ اجرای آن‌ها در ترتیب اشتباه نتایج گمراه‌کننده می‌دهد.

ماتریس پذیرش

شماره / عنوان	ابزار اثبات	وضعیت نیازمند بررسی
1 / Shard distribution	Stage03 + Stage14 audit	لاگ واقعی
2 / Replica load	Stage04 normal / failover	لاگ واقعی
3 / Concurrent local writes	Stage05	لاگ واقعی
4 / Replica interruption	Stage04 a1-down + Stage08 healed	لاگ و Container ID
5 / Local + cross transfer	Stage05 + Stage06 / Stage13	تأیید 2PC
6 / Coordinator interruption	Stage07 crashed/recovered	تصمیم و ACK پایدار
7 / Concurrent cross-shard	Stage13 overlap	حفظ پول و Ledger
Bonus I	Stage10–14 + 4 crash points	پیشرفت Cutover
Bonus II	Stage08/09 four controller faults	نبود Duplicate

ثبت دقیق یافته فعلی

اجرای واقعی دستور verify_stage14 --scenario=postcutover --json با AssertionError متوقف شد و active_ranges را به صورت A:1–1000، B:1001–2000 گزارش کرد. این پیام به معنی خراب‌شدن حساب‌ها نیست، بلکه نشان می‌دهد شرط وضعیت نهایی بعد از Cutover هنوز برقرار نشده است. لذا این تست نباید در ویدیو PASS معرفی شود تا مراحل 11/12/13 با ترتیب صحیح کامل شوند.

جدول خروجی‌ها باید پس از هر اجرای Docker با تاریخ و لینک لاگ تکمیل شود؛ گزارش حاضر نتیجه‌ای را که مشاهده نشده جعل نمی‌کند.

10 | جمع‌بندی، محدودیت‌ها و راهنمای تحویل

جمع‌بندی فنی

این پروژه چارچوب یک سامانه بانکی توزیع‌شده روی یک میزبان را ارائه می‌دهد: مالکیت قطعی حساب‌ها، توزیع درخواست میان Replicaهای سالم، کنترل هم‌زمانی با 2PC، PostgreSQL پایدار، بازیابی تصمیم، مهاجرت زنده با Journal و Snapshot، تغییر مالکیت با Epoch و Fence، و Self-Healing سفارشی. معیار موفقیت نهایی فقط وجود سورس نیست؛ اجرای قابل تکرار همه سناریوها و ثبت شواهد عملی است.

محدودیت‌های شناخته‌شده

- Single Host: خرابی کامل Docker Host یا Metadata DB از محدوده بازیابی اعلام‌شده خارج است.
- Prepared 2PC ممکن است در شرایط خرابی طولانی Block شود؛ این محدودیت ذات پروتکل است.
- Migration طراحی‌شده برای یک انتقال B→C است؛ چارچوب عمومی چندباره Migration در این گزارش ادعا نشده است.
- کنترل امنیتی Docker Socket برای محیط تولید کافی نیست و نیازمند جداسازی بیشتر است.
- حذف داده قدیمی B نباید پیش از تأیید کامل صحت و تکمیل Cleanup انجام شود.

بسته تحویل

وضعیت / محل تکمیل	قلم تحویلی
لینک واقعی Repository و Tag stage-15 درج شود	GitHub Source
نام واقعی Imageها و Tagهای Pushشده درج شود	Docker Hub Images
همین گزارش 10 صفحه‌ای، پس از تکمیل نتایج	Technical Report
حداکثر 15 دقیقه، براساس فایل TXT تست‌ها	Video Demo
لاگ 7 آزمایش و 2 Bonus همراه timestamp و شناسه	Evidence

گام ضروری پیش از اعلام تحویل نهایی

در محیط جاری ابتدا وضعیت Migration Job بررسی شود؛ اگر مرحله CATCHING_UP آماده بود، مرحله preflight 12 انجام گیرد. تنها بعد از PASS و پشتیبان‌گیری، Cutover یک‌بار اجرا شود، آرشیو request_idهای B تکمیل گردد و مرحله 14 دوباره اجرا شود. سپس نتایج واقعی در گزارش ثبت و برچسب Git نهایی ایجاد شود. audit

منبع الزام‌ها: Assignment 2 — Distributed Banking Platform، چهار صفحه، بخش‌های 1 تا 9؛ منبع پیاده‌سازی: سورس و مستندات مراحل 00 تا 14 پروژه.